

CODEALPHA CYBER SECURITY INTERNSHIP

TASK 1: BASIC NETWORK SNIFFER

Submitted By: Meerab Asif

GitHub Repository: https://github.com/fa-2023-bscs-501-design/CodeAlpha_BasicNetworkSniffer

1. Introduction

Network sniffing is the process of capturing and monitoring data packets transmitted over a computer network. This project develops a Basic Network Sniffer using Python and the Scapy library to capture live packets and display source IP, destination IP, protocol, ports, and packet length.

2. Objectives

- Understand packet sniffing
- Capture live packets
- Analyze network traffic
- Learn Scapy
- Improve cybersecurity skills

3. Software and Tools Used

Python 3.13

Visual Studio Code

Scapy

Npcap

Git

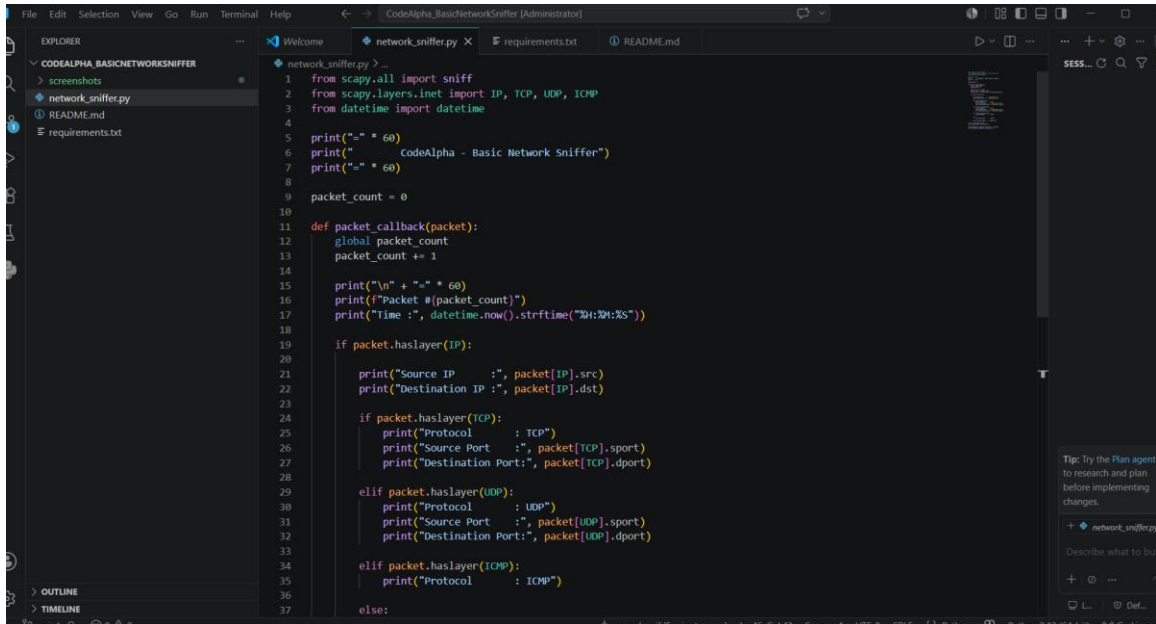
GitHub

4. Project Description

The application captures live packets using Scapy and displays source IP, destination IP, protocol type, source port, destination port and packet length in real time.

5. Working Methodology

The sniff() function captures packets. The program checks the IP layer, extracts information, identifies TCP/UDP/ICMP protocols and prints the details.



```
1 from scapy.all import sniff
2 from scapy.layers.inet import IP, TCP, UDP, ICMP
3 from datetime import datetime
4
5 print("-" * 60)
6 print("      CodeAlpha - Basic Network Sniffer")
7 print("-" * 60)
8
9 packet_count = 0
10
11 def packet_callback(packet):
12     global packet_count
13     packet_count += 1
14
15     print("\n" + "-" * 60)
16     print(f"Packet #{packet_count}")
17     print("Time :", datetime.now().strftime("%H:%M:%S"))
18
19     if packet.haslayer(IP):
20
21         print("Source IP      :", packet[IP].src)
22         print("Destination IP :", packet[IP].dst)
23
24         if packet.haslayer(TCP):
25             print("Protocol      : TCP")
26             print("Source Port   :", packet[TCP].sport)
27             print("Destination Port:", packet[TCP].dport)
28
29         elif packet.haslayer(UDP):
30             print("Protocol      : UDP")
31             print("Source Port   :", packet[UDP].sport)
32             print("Destination Port:", packet[UDP].dport)
33
34         elif packet.haslayer(ICMP):
35             print("Protocol      : ICMP")
36
37     else:
```

6. Features

- Live packet capture
- Source/Destination IP
- TCP/UDP/ICMP detection
- Port information
- Packet length
- Real-time monitoring

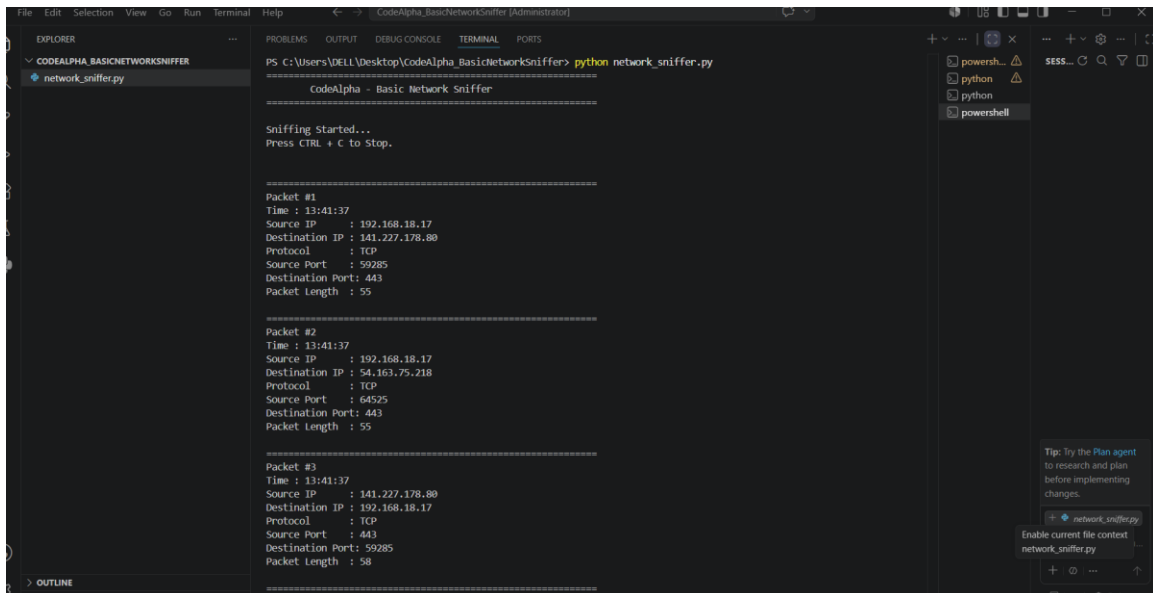
7. Installation

1. Install Python
2. Install Npcap
3. pip install scapy
4. Run: python network_sniffer.py

8. Output

The program successfully captured live packets.

Output



```
PS C:\Users\DELL\Desktop\CodeAlpha_BasicNetworkSniffer> python network_sniffer.py

CodeAlpha - Basic Network Sniffer

=====

Sniffing Started...
Press CTRL + C to Stop.

=====

Packet #1
Time : 13:41:37
Source IP : 192.168.18.17
Destination IP : 141.227.178.80
Protocol : TCP
Source Port : 59285
Destination Port: 443
Packet Length : 55

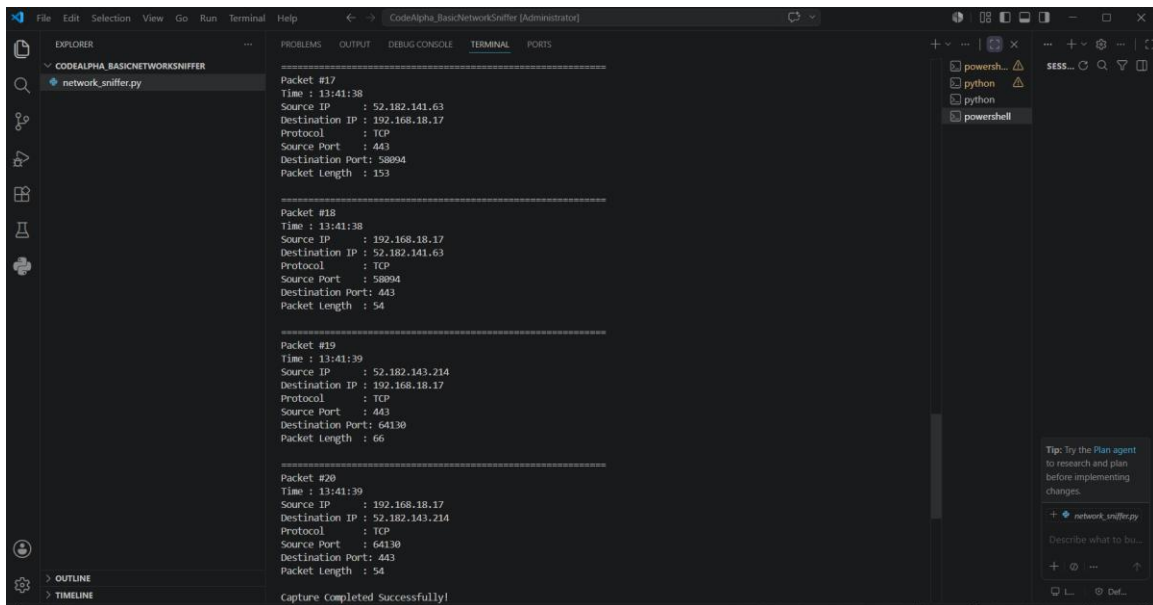
=====

Packet #2
Time : 13:41:37
Source IP : 192.168.18.17
Destination IP : 54.163.75.218
Protocol : TCP
Source Port : 64525
Destination Port: 443
Packet Length : 55

=====

Packet #3
Time : 13:41:37
Source IP : 141.227.178.80
Destination IP : 192.168.18.17
Protocol : TCP
Source Port : 443
Destination Port: 59285
Packet Length : 58

=====
```



```
Packet #17
Time : 13:41:38
Source IP : 52.182.141.63
Destination IP : 192.168.18.17
Protocol : TCP
Source Port : 443
Destination Port: 58094
Packet Length : 153

=====

Packet #18
Time : 13:41:38
Source IP : 192.168.18.17
Destination IP : 52.182.141.63
Protocol : TCP
Source Port : 58094
Destination Port: 443
Packet Length : 54

=====

Packet #19
Time : 13:41:39
Source IP : 52.182.143.214
Destination IP : 192.168.18.17
Protocol : TCP
Source Port : 443
Destination Port: 64130
Packet Length : 66

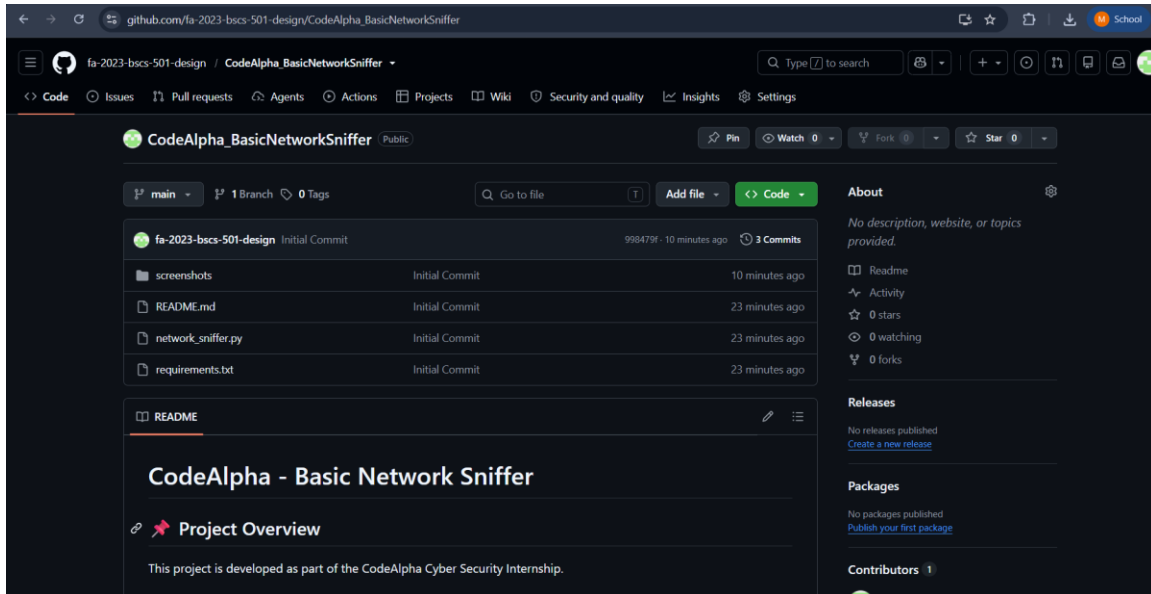
=====

Packet #20
Time : 13:41:39
Source IP : 192.168.18.17
Destination IP : 52.182.143.214
Protocol : TCP
Source Port : 64130
Destination Port: 443
Packet Length : 54

=====

Capture Completed Successfully!
```

GitHub Repository



9. Learning Outcomes

- Packet Sniffing
- TCP/IP Basics
- Python Programming
- Network Analysis
- GitHub Usage

10. Conclusion

This project provided hands-on experience in packet sniffing, network analysis and cybersecurity fundamentals using Python and Scapy.